Student notes — Session 1 · Course intro and your first C program

Slides: [`Week_01_Programming_Principles_I_C.pdf`](../slides/Week_01_Programming_Principles_I_C.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Title

CSCI 1101, Programming Principles I, Fall 2026. This course teaches you to write programs in the C language, starting from nothing. You are not expected to have written code before.

When and where. Tuesday in B102 and Friday in B101, both 16:00 to 17:15. Note that the two rooms are different.

Where everything lives. Blackboard: slides, the syllabus, homework, grades, announcements.

How to ask a question. Use the Blackboard discussion forum first. If you are confused, other people are confused about the same thing, and on the forum everyone sees the answer. For private matters, email ashaliyev@ada.edu.az. Office hours are by appointment only — book a slot with the link in the syllabus. Please do not knock on B017 without a booking.

How you are graded. Quizzes 15%, Project 1 10%, Project 2 10%, Project 3 15%, Midterm 20%, Final 30%.

- **Quizzes:** six short ones, about every second week, one or two short questions each. Only your **best five count**, so one bad day costs you nothing — and that is why there are no make-up quizzes.
- **Projects:** three programs, getting harder as the semester goes. After you submit, the Teaching Assistant may ask you to explain part of your code or make a small change to it. That counts towards the project grade, so never submit code you cannot explain.
- **Midterm and final:** both on paper. The midterm covers the first half; the final covers everything, with more weight on the second half.

Knowledge checks (0%). Every week a short check opens on Blackboard: about six multiple-choice questions on that week. It is worth zero percent and **cannot lower your grade**. You get three attempts and the best one is kept, and each check stays open for three weeks. They exist so you can find out what you did not understand while there is still time to ask, and they are the best preparation for the quizzes, which cover the same material without a laptop. Each check you complete earns a Blackboard badge. Students who complete every check on time may receive small bonuses at the instructors' discretion.

Attendance. By law you must attend at least 75% of classes or you fail the course. Attendance is taken at the **start** of class from week 2, right after Add/Drop closes on 16 September. There is no late mark. If you are ill, send documents to your advisor.

Deadlines are never extended. Up to 30 minutes late costs 10%; up to a day, 25%; each further day, another 25%.

What you need on your laptop. An editor and a C compiler. We recommend **Visual Studio Code** with the **C/C++ extension**, both free. Any C compiler is fine. On Windows use the one-click installer we provide, or WSL, or the Microsoft compiler; on macOS run `xcode-select --install`; on Linux install GCC from your package manager. If you cannot install anything, use **CoCalc** (https://cocalc.ai/) in your browser. Step-by-step instructions are in "Setting up a C compiler" on Blackboard. Bring your laptop to every class.

Slide 2 — Today's roadmap

Six things for week 1: set up the tools, write a small C program, compile it from the terminal, use variables, read input and print output, and use basic math functions.

The program on the right of the slide is where you will be by the end of the week. It reads a radius from the user and prints the area of the circle. Right now it probably looks like nonsense. By Friday you will understand every character of it.

The sentence at the bottom is the most important one: **programming is not memorizing syntax, it is translating a problem into precise steps a machine can follow**. The typing is easy. The thinking is the course.

Slide 3 — What is a program?

A program is a list of instructions for the computer. The computer follows them exactly. It does what you **wrote**, not what you **meant**.

Example. A recipe says "boil the eggs". A person knows what that means. A computer does not. For a computer you have to write: put water in the pot, put the pot on the stove, turn the stove on, wait until the water boils, put the eggs in, wait eight minutes, take the eggs out. Every step. In order.

Programming has three stages. Do not skip the middle one.

1. **The task in words.** "Calculate the area of a circle."
2. **The steps, in plain language.** Ask for the radius. Multiply pi by the radius by the radius. Show the result. This list of steps is called an **algorithm**. There is no C in it. You can write it on paper.
3. **The C code.** Only now you turn each step into C.

The question on the slide. "Calculate the salary." What is missing? A computer cannot do this, because it does not know: how many hours were worked, how much per hour, is there tax, is there a bonus. Before you can write a program, every number it needs and every step it takes must be written down.

Slide 4 — From source code to running program

The computer cannot read C. C is a language for people. So there are three separate things, and it is important to keep them apart.

1. **Your file.** It ends in `.c`, for example `hello.c`. It is text that you typed. You can open it and read it. The computer **cannot run it**.
2. **The compiler.** That is the program called `gcc`. It reads your `.c` file and translates it into a form the computer can run.
3. **The program.** That is the new file `gcc` makes. On Windows it is `hello.exe`, on Mac and Linux just `hello`. **This** is the file that runs.

The two commands. Type them in the terminal, one at a time, and press Enter after each.

```
gcc -Wall hello.c -o hello
./hello
```

- The first line says: `gcc`, take `hello.c`, and put the result in a file called `hello`. `-o` means "output". `-Wall` means "warn me about everything that looks wrong".
- The second line runs the program. `./` means "the one in this folder". In Windows PowerShell type `.\hello` instead.

What you see. After the first command, if everything is right, you see **nothing**. The terminal just shows a new line. That is good. Silence means success. After the second command you see what your program prints.

When you see red text instead. That is an error. Everyone gets errors. Your instructor gets errors. The compiler is telling you the file name, the line number, and what it did not understand. Read it. Fix one thing. Compile again.

Slide 5 — Week 1 setup checklist

What you need on your laptop:

- **Visual Studio Code**, called VS Code. This is where you type your program. It is free.
- **A C compiler**, such as `gcc`. This is what turns your program into something the computer can run.
- **The C/C++ extension** for VS Code. It colours your code and underlines mistakes. Free too.
- **A terminal.** The window where you type commands. VS Code has one built in: click **View**, then **Terminal**.
- **Git.** Not needed yet. Skip it.

How to install all of this is in "Setting up a C compiler" on Blackboard, step by step.

Your test. Type the small program on the slide into VS Code. Save it as `test.c`. In the terminal, type:

```
gcc -Wall test.c -o test
./test
```

If the screen shows `C setup works!`, you are ready. If you see an error instead, bring your laptop to class with the error still on the screen. We fix it together.

Slide 6 — Command line basics

The terminal is a window where you type commands instead of clicking. It looks scary. It is not. In this course you need only six commands.

You want to	Type	Example
know which folder you are in	`pwd`	`pwd` shows `/home/ali`
see the files in this folder	`ls` (on Windows also `dir`)	`ls` shows `hello.c`
go into a folder	`cd` and the folder's name	`cd c_week1`
go back up one folder	`cd ..`	`cd ..`
make a new folder	`mkdir` and a name	`mkdir c_week1`
compile	`gcc -Wall` file `-o` name	`gcc -Wall hello.c -o hello`
run	`./` and the name	`./hello`

A walk through, so you see what happens. You open a terminal. You type:

1. `pwd` and Enter. The screen shows your home folder, something like `C:\Users\ali` or `/home/ali`.
2. `mkdir c_week1` and Enter. Nothing is shown. The folder was made.
3. `ls` and Enter. Now `c_week1` is in the list.
4. `cd c_week1` and Enter. Nothing is shown. You are inside now.
5. `pwd` and Enter. The screen shows a path ending in `c_week1`. See, you moved.
6. `ls` and Enter. Nothing. The folder is empty. You just made it.
7. `cd ..` and Enter. You are back where you started.

The most common problem of the first month. You type `gcc hello.c -o hello` and the screen says `hello.c: No such file or directory`. This almost always means **you are in the wrong folder**. `gcc` looks only in the folder you are in. Type `pwd` to see where you are. Type `ls` to see what is there. If `hello.c` is not in the list, `cd` to the folder where it is.

Do this at home. Make a folder called `c_week1`. Save `hello.c` inside it. Compile it. Run it.

Slide 7 — First C program: anatomy

Here is a complete C program. Every program you write this semester has this shape.

```
#include <stdio.h>
int main(void) {
    printf("Hello, ADA!\n");
    return 0;
}
```

When you compile and run it, the screen shows:

```
Hello, ADA!
```

Line by line.

- `#include <stdio.h>`. Say it "hash include standard i-o". This line gives your program the ability to print. Without it, `printf` does not exist. For now, always start your program with this exact line. You will understand it fully in week 8.
- `int main(void) {`. This is where the program **starts**. When you run a program, the computer looks for `main` and begins there. Every program has exactly one `main`. The words `int` and `void` are explained in week 4. For now copy this line exactly.
- `{` and `}`. Everything between the curly braces belongs to `main`. They come in pairs. If you open one, you must close one.
- `printf("Hello, ADA!\n");`. This prints. What is between the quotes goes on the screen. The `\n` at the end is not shown: it means "go to a new line". Note it is a backslash `\`, the one that leans to the left, not `/`.
- `return 0;`. This says "the program is finished and everything is fine". Zero means fine. Yes, that feels backwards.

Semicolons. Every instruction inside `main` ends with `;`. That is how C knows where one instruction stops. The `#include` line, the `int main(void)` line and the braces do **not** get a semicolon.

Example of the most common error. Remove the semicolon after `printf("Hello, ADA!\n")` and compile. The screen shows something like:

```
hello.c:3:28: error: expected ';' before 'return'
```

Read it: file `hello.c`, line 3, it expected a semicolon. Put it back, compile again, and the error is gone. This is the normal way of working: type, compile, read the error, fix one thing, compile again.

Try it yourself. Change the message to your own name. Compile. Run. Then add a second `printf` line under the first, with a different message, and run again. You should see two lines.

Slide 8 — Variables: named boxes in memory

A **variable** is a box in the computer's memory. It has a name and something inside it.

```
int age = 20;
```

This makes a box called `age` and puts 20 in it. After this line, whenever you write `age`, C looks in the box and uses what is inside.

Two steps, which you can do together or apart.

- **Making the box** is called declaring. `int count;` makes a box called `count` for whole numbers. It is empty for now.
- **Putting something in** is called assigning. `count = 5;` puts 5 in the box.
- Usually you do both at once: `int count = 5;`.

You make a box **once**. You can put a new value in it as many times as you like. The old value is thrown away each time. The box holds one value.

```
int count = 5;
count = 8;
count = 2;
printf("%d\n", count);
```

The screen shows `2`. The 5 and the 8 are gone.

Types. When you make a box, you must say what kind of thing goes in it. Three types for this week:

Type	Holds	Examples	Use it for
`int`	whole numbers	`20`, `-3`, `0`	age, number of students, a count
`double`	numbers with a decimal point	`4.75`, `3.14159`, `1.8`	price, height, average
`char`	one single character	`'A'`, `'z'`, `'7'`	a letter grade, an answer y/n

Examples of making each kind:

```
int students = 30;
double price = 4.75;
char grade = 'A';
```

Quotes. One character uses **single** quotes: `'A'`. Text uses **double** quotes: `"Hello"`. They are not the same to C. `'A'` is a `char`. `"A"` is text.

One surprise, explained fully on Friday. If you divide two whole numbers, C throws away the decimal part. `7 / 2` is `3`, not `3.5`. For now, if a value can have a decimal point, use `double`.

Try it yourself. Make a variable for your age, one for your height, and one for the first letter of your name. Print all three with labels.

Slide 9 — The assignment trap

```
x = x + 1;
```

In school maths this line is nonsense. No number equals itself plus one. So do not read it as maths.

In C, `=` does not mean "is equal to". It means: **work out the right side, then put the answer in the box on the left.** Read it from right to left.

`x = x + 1;` means: take what is in `x`, add 1, put the result back in `x`.

Trace it step by step.

```
int x = 5;
x = x + 1;
printf("%d\n", x);
```

1. `int x = 5;` makes the box `x` and puts 5 in it. `x` is 5.
2. `x = x + 1;` First the right side. What is in `x`? 5. Add 1. That is 6.
3. Now 6 is put into `x`. The 5 is gone. `x` is 6.
4. `printf` shows `6`.

A second one.

```
int y = 3;
y = y * 2 + 1;
printf("%d\n", y);
```

Right side first. `y` is 3. Multiply before you add: 3 × 2 = 6. Then 6 + 1 = 7. Put 7 in `y`. The screen shows `7`. If you got 8, you added first. Multiplication comes before addition, as in school.

A third one, with two boxes.

```
int a = 10;
int b = a + 5;
a = 1;
printf("%d %d\n", a, b);
```

`b` becomes 10 + 5 = 15. Then `a` is changed to 1. `b` does not change, because the calculation for `b` already happened. The screen shows `1 15`.

The rule, once more: right side first, then the answer goes into the box on the left. Almost everything in this course uses this. If it is still unclear, ask in the forum or in class.

Try it yourself. Before you run each program, write on paper what you think it prints. Then run it.

1. `int n = 10; n = n - 3; printf("%d\n", n);` (Expect `7`.)
2. `int n = 2; n = n * n; printf("%d\n", n);` (Expect `4`.)
3. `int n = 1; n = n + 1; n = n + 1; printf("%d\n", n);` (Expect `3`.)

Slide 10 — Standard output: printf

A program that calculates something but never says anything is useless. `printf` is how a program tells you what it found.

The simple form prints fixed text: `printf("Hello!\n");`

The useful form prints what is inside a variable:

```
printf("Age: %d\n", age);
```

There are two parts, separated by a comma: the text in quotes, then the variable. Inside the text, `%d` is a **hole**. It is not printed — it is **replaced** by the value of the variable. So the screen shows: `Age: 20`.

The hole must match the type of the variable:

- `%d` — for an `int`
- `%f` — for a `double`
- `%.2f` — for a `double` shown with exactly two digits after the point. Use this for money.
- `%c` — for a single `char`
- `%s` — for text; we cover it in week 6.

From the three lines on the slide: `%d` with `age` prints `Age: 20`; `%.2f` with `gpa` prints `GPA: 3.75`; `%c` with `letter` prints `Grade: A`. The `%.2f` line is the one that prints exactly two decimals.

If you use the wrong specifier for the type, you usually get a compiler warning, and if you ignore it you get meaningless numbers on the screen. **When your output looks insane, check your `%` letters first.**

A habit worth keeping all semester: always print a label, never a bare number. Print `Age: 20`, not `20`. When a program prints five bare numbers and one of them is wrong, you cannot tell which is which.

Before the next class

1. **Install your compiler**, following "Setting up a C compiler" on Blackboard.
2. **Make a folder, write `hello.c`, compile it, run it.** Then change the message and do it again. Fifteen minutes now saves you an hour later.
3. **Bring your laptop to Friday's class, in B101.** If your setup does not work, bring it anyway with the error on the screen — we fix it in class. Nobody is behind.

Friday covers `scanf` (how a program asks *you* for a number) and the math functions.